

## RELATÓRIO TÉCNICO & DE QUALIDADE

# CashWise

Code Review · Carga de Teste · Regras de Negócio · Backlog · Recomendações

Preparado por: Tech Lead / Análise de Qualidade Sênior (sessão assistida por Claude)

Destinatário: Luis Gustavo do Nascimento Marinho

Data: 06 de setembro de 2026 — Branch: `develop`

## Sumário

---

1. Resumo executivo
2. Panorama — o que funciona vs. o que não funciona
3. Code review — achados
4. Carga de teste
5. Regras de negócio identificadas
6. Tarefas pendentes
7. Sugestões e melhorias
8. Próximos passos recomendados

## 1. Resumo executivo

O CashWise saiu, ao longo desta série de sessões, de um estado de **protótipo com backend e frontend desconectados** (nenhuma autenticação real, dados fictícios em todas as telas) para uma aplicação com **jornada completa funcional**: cadastro, login com JWT, isolamento de dados por usuário, dashboard calculado a partir de dados reais, CRUD de transações e metas financeiras, tudo coberto por 29 testes automatizados e rodando via Docker.

Esta rodada de auditoria teve um objetivo diferente das anteriores: não é mais "o que falta construir", é "**o que já existe está correto, aguenta carga, e segue as próprias regras que deveria seguir?**". A resposta curta: a fundação (autenticação, isolamento, infraestrutura) está sólida e passou nos testes de carga sem erros. Mas o code review desta rodada encontrou **falhas reais de validação de regra de negócio** que não tinham sido pegas antes — nenhuma delas de segurança, mas todas capazes de corromper silenciosamente os números que o usuário vê no dashboard.



## 2. Panorama — o que funciona vs. o que não funciona

### Backend

| Recurso                              | Status   | Observação                                                      |
|--------------------------------------|----------|-----------------------------------------------------------------|
| Cadastro / Login (JWT)               | Funciona | Hash BCrypt, mensagens de erro não vazam qual campo está errado |
| Autorização / isolamento por usuário | Funciona | Testado automaticamente (usuário A não acessa dado de B)        |
| Transações (CRUD)                    | Funciona | Isolado, mas <b>aceita valores negativos/zero</b> (achado nº1)  |

|                          |                |                                                                                |
|--------------------------|----------------|--------------------------------------------------------------------------------|
| Metas Financeiras (CRUD) | Funciona       | Mesmo problema de validação de valor                                           |
| Dashboard/resumo         | Funciona*      | *Cálculo correto, mas herda a corrupção dos achados nº1 e nº2                  |
| Troca de dados/senha     | Funciona       | Senha atual é verificada antes de trocar                                       |
| Conteúdo educativo       | Parcial        | Leitura pública funciona; tabela vazia; sem dono real do conteúdo (achado nº3) |
| CI (GitHub Actions)      | Não verificado | Só valida de fato no primeiro push real                                        |

Frontend

| Página                        | Status  | Observação                                                                  |
|-------------------------------|---------|-----------------------------------------------------------------------------|
| Login / Cadastro              | Real    | Conectadas à API                                                            |
| Dashboard (Página do Usuario) | Real    | Dados reais; herda os bugs de cálculo do backend                            |
| Metas Financeiras             | Real    | CRUD completo validado                                                      |
| Alterar Usuario               | Real    | Nome/e-mail/senha salvam de verdade                                         |
| Perfil Usuario                | Parcial | Campos fictícios (idade, período) sem correspondência no backend            |
| Alterar Administrador         | Fake    | Todo botão é um <code>alert()</code> ; não existe papel de admin no backend |

3. Code review — achados

Achado 1 — Transações e metas aceitam valores negativos ou zero

ONDE

`TransacaoRequest.Valor` e `MetaFinanceiraRequest.Valor` (Dtos) são `double` marcados com `[Required]` — que, para tipos numéricos não anuláveis, não valida nada de fato.

REPRODUZIDO EM

POST /api/Transacao COM {"valor": -999, "tipo": "Receita"} → **201 Created**, e o valor negativo entrou na soma de `totalReceitas` do dashboard, distorcendo o saldo exibido.

#### IMPACTO

Um erro de digitação do usuário (ou um clique duplo mal tratado no futuro) corrompe silenciosamente o saldo mostrado, sem nenhum aviso.

#### CORREÇÃO RECOMENDADA

Trocar `[Required]` por `[Range(0.01, double.MaxValue)]` nos dois DTOs.

### Achado 2 — "Tipo" de transação é texto livre e some silenciosamente do dashboard se não for exatamente "Receita" ou "Despesa"

#### ONDE

`DashboardController.GetResumo()` filtra transações comparando `Tipo` contra as strings literais "Receita"/"Despesa". Nada no DTO nem no banco impede outro valor.

#### REPRODUZIDO EM

POST /api/Transacao COM "tipo":"Investimento" → 201 Created, aparece normalmente em GET /Transacao, mas **desaparece completamente** de `totalReceitas`, `totalDespesas` e `saldoTotal` no resumo.

#### IMPACTO

O usuário vê a transação na lista, mas ela não é contada em lugar nenhum — inconsistência visível entre a tabela e os cards de resumo.

#### CORREÇÃO RECOMENDADA

Trocar `Tipo` por um `enum` (`Receita`, `Despesa`) validado na entrada, já mapeado no backlog como CW2-021 mas ainda não priorizado.

### Achado 3 — Conteúdo educativo: qualquer usuário autenticado pode editar/excluir conteúdo de qualquer outro, e o "dono" (UsuarioFk) é definido pelo próprio cliente

#### ONDE

`ConteudoController` — `CreateConteudo` / `UpdateConteudo` aceitam a entidade `Conteudo` crua, incluindo `UsuarioFk`, sem checar se corresponde ao usuário autenticado;

`UpdateConteudo` / `DeleteConteudo` não verificam posse do registro.

#### IMPACTO

Hoje de baixo risco prático (tabela vazia, sem uso real ainda), mas é uma regra de negócio que nunca foi decidida conscientemente — "qualquer autor pode editar qualquer artigo"

pode ser aceitável para um blog colaborativo, mas precisa ser uma decisão, não um acidente de implementação.

**CORREÇÃO RECOMENDADA**

Decidir o modelo de posse do conteúdo antes de popular a tabela; setar `UsuarioFk` a partir do token, nunca do corpo da requisição.

**Achado 4 — Exclusão de usuário deixa registros órfãos em CONTEUDO**

**ONDE**

`Conteudo.UsuarioFk` é um `int` solto, sem propriedade de navegação — não existe FK real no banco (confirmado na migration gerada). `Transacao` e `MetaFinanceira` têm cascade delete configurado; `Conteudo` não.

**IMPACTO**

`DELETE /api/Usuario/me` apaga o usuário mas deixa linhas em `CONTEUDO` apontando para um `USUARIOFK` que não existe mais.

**CORREÇÃO RECOMENDADA**

Adicionar propriedade de navegação `Usuario` em `Conteudo.cs` e configurar a FK/cascade no `ApplicationDbContext`, junto com a correção do Achado 3.

**4. Carga de teste**

Executada com `autocannon` contra a API rodando em Docker (container único, sem réplicas, MySQL no mesmo host) — resultados são indicativos de comportamento sob concorrência, não uma medição de capacidade de produção.

| Endpoint                                     | Carga               | Latência p50 / p99 | Throughput                     | Erros |
|----------------------------------------------|---------------------|--------------------|--------------------------------|-------|
| <code>GET /Transacao</code><br>(autenticado) | 50 conexões,<br>30s | 33 ms / 81 ms      | ~1.410 req/s (42 mil no total) | 0     |
| <code>POST /Auth/login</code>                | 20 conexões,<br>20s | 607 ms / 989 ms    | ~32 req/s (658 no total)       | 0     |
| <code>POST /Transacao</code> (escrita)       | 25 conexões,<br>15s | 51 ms / 118 ms     | ~455 req/s (7.000 no total)    | 0     |

## Análise

- **Leitura e escrita autenticadas se comportam bem** sob concorrência moderada — sem erros, sem timeouts, sem escritas perdidas (7.000 requisições de escrita concorrentes geraram exatamente 6.850 linhas novas, consistente com o throughput medido nos 15s de teste).
- **Login é ~44x mais lento que leitura simples — isso é esperado e intencional:** o BCrypt foi desenhado para ser computacionalmente caro (resiste a força bruta). 32 logins/segundo sustentados equivalem a mais de 100 mil logins/hora, muito acima do que este produto precisa hoje. Não é um problema a corrigir — é o comportamento correto de um hash de senha.
- **Não testado nesta rodada:** comportamento sob falha do MySQL (connection pool exhaustion), comportamento com volume de dados muito maior (paginação ainda não existe em nenhuma listagem), e teste de carga através do Docker completo (frontend + API + banco simultaneamente).

## 5. Regras de negócio identificadas

---

Regras que o código hoje implementa — extraídas por leitura direta dos controllers e DTOs, não de documentação (que não existe formalmente para o domínio):

1. Um usuário só pode ver, alterar ou excluir seus próprios dados (transações, metas, perfil) — **implementada e testada.**
2. E-mail é único no sistema, verificado tanto na aplicação quanto por índice único no banco — **implementada.**
3. Senha exige no mínimo 8 caracteres, tanto no cadastro quanto na troca de senha — **implementada.**
4. Sessão expira 60 minutos após o login, sem renovação automática (sem refresh token, por decisão explícita) — **implementada.**
5. Saldo = soma de receitas – soma de despesas do usuário — **implementada, mas vulnerável aos achados 1 e 2.**
6. Gastos do mês = soma de despesas cuja data cai no mês/ano correntes — **implementada.**
7. Nome de meta financeira pode se repetir entre usuários diferentes (corrigido um bug herdado do schema original que impedia isso globalmente) — **implementada.**
8. Transação deve ser do tipo "Receita" ou "Despesa" — **nunca foi de fato implementada como restrição**, apesar de estar em comentários no código (achado 2).

- 9. Valor de transação/meta deve ser positivo — **nunca foi implementada** (achado 1).
- 10. Quem pode editar/excluir um conteúdo educativo — **nunca foi decidida** (achado 3).
- 11. Existência de um papel "administrador" com permissões especiais — **existe na intenção da UI (tela Alterar Administrador) mas nunca foi modelada no backend**. Não há campo de role/permissão em `Usuario`.

## 6. Tarefas pendentes

| Prioridade | Tarefa                                                                                               | Motivo                                                                     |
|------------|------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------|
| P0         | Corrigir Achados 1 e 2 (validação de Valor e Tipo)                                                   | Corrompem dado exibido ao usuário, baixo esforço de correção               |
| P1         | Decidir e corrigir posse de Conteúdo (Achados 3 e 4)                                                 | Antes de popular a tabela com conteúdo real                                |
| P1         | Simplificar Perfil Usuario para campos reais (Nome/E-mail)                                           | Decisão pendente há duas sessões — recomendado executar sem mais adiamento |
| P2         | Popular Conteúdo com dados reais e conectar a página à API                                           | Feature existe nos dois lados mas nunca foi usada de ponta a ponta         |
| P2         | Decidir o futuro de "Alterar Administrador": implementar papel de admin de verdade ou remover a tela | Hoje é 100% simulado, sem lastro no backend                                |
| P2         | Paginação em listagens (Transacao, MetaFinanceira)                                                   | Load test não cobriu volume alto de dados por usuário                      |
| P3         | Verificar o workflow de CI com um push real                                                          | Nunca foi validado em execução real, só localmente                         |
| P3         | Configuração de ambiente para <code>API_BASE_URL</code> (hoje hardcoded)                             | Bloqueador para qualquer deploy fora de localhost                          |

## 7. Sugestões e melhorias

---

### Curto prazo (baixo esforço, alto retorno)

- Os Achados 1 e 2 são consertos de poucas linhas ( `[Range]` nos DTOs, `enum` para Tipo) com testes automatizados já prontos como padrão a seguir — recomendo tratá-los antes de qualquer feature nova.
- Adicionar um teste de regressão para cada achado desta auditoria, para que nenhum deles volte a acontecer silenciosamente.

### Médio prazo

- Formalizar as regras de negócio da seção 5 em um documento curto (mesmo que informal) — hoje elas só existem implicitamente no código, o que torna fácil quebrá-las sem perceber (como os achados desta auditoria mostram).
- Avaliar se "papel de administrador" é realmente necessário para o objetivo do produto antes de investir na tela — se não for, considerar remover "Alterar Administrador" em vez de deixá-la fake indefinidamente.

### Mais à frente

- Uma vez que Transação/Meta tenham validação de valor e tipo confiáveis, a base estará pronta para a camada de IA planejada (chat financeiro, categorização) sem herdar dado corrompido.
- CI real (rodar em um push de verdade) antes de qualquer trabalho de deploy em produção.

## 8. Próximos passos recomendados, em ordem

---

1. **Corrigir Achados 1 e 2** — validação de valor e tipo nas transações/metadados, com testes cobrindo os casos negativo/zero/tipo-inválido.
2. **Simplificar Perfil Usuario** — decisão já tomada nesta auditoria: mostrar só Nome/E-mail reais.
3. **Decidir e corrigir a posse de Conteúdo** (Achados 3 e 4) antes de popular dados reais nessa feature.

CashWise — Relatório gerado em sessão assistida · 06/09/2026